C++ 本地环境配置：VSCode / CLion

根据是否已经完成 VSCode 配置，选择下面的一条路径。两条路径均使用 C++17，并以运行 Hello World 为完成标准。课堂继续使用教师提供的 CPP-Classroom。

English · 完整离线 PDF · Hello World 源码

方法一：已有 VSCode

保留现有 VSCode 配置，按系统补齐编译器：

- **Windows**：参考配置视频及指南中的 MSYS2 安装和 PATH 配置步骤。

- **macOS**：执行 `xcode-select --install`，安装 Xcode Command Line Tools。g++ 调用 Apple Clang，可用于本课程的 C++17 程序。

- **Linux**：先执行 `g++ -v`；已有编译器则直接使用，缺少时按发行版安装。指南提供 Ubuntu/Debian 和 Fedora 命令。

完成标准：在 VSCode 集成终端（Terminal）中执行 `g++ -v`，再用 `g++ -std=c++17` 编译并运行 Hello World。

VSCode 中文指南 · 中文 PDF · English guide · English PDF

方法二：尚未配置 VSCode

沿用原来的 **CLion + 学校邮箱申请的 Student Pack 学生授权**方案。按指南完成申请、安装和激活；Windows 使用 Bundled MinGW，macOS 使用 Xcode Command Line Tools 提供的 Apple Clang。

新建 C++ Executable 项目，语言标准选择 C++17，将附带的 `hello.cpp` 复制到 `main.cpp`，点击 Run。

完成标准：在 CLion 的 Run 窗口看到 `Hello World`。

CLion 中文指南 · 中文 PDF · English guide · English PDF

Linux/Ubuntu 使用 CLion 的同学可以继续使用 Linux 课程环境，执行 `sudo apt update`、`sudo apt install build-essential gdb cmake` 安装工具链，再用 CLion 打开本模块的 CMake 项目。其他构建方式见模块 README。

完整离线 PDF 依次包含本页、VSCode 指南和 CLion 指南。

返回 C++ 课程

VSCode 本地 C++ 环境配置（Windows / macOS / Linux）

已有 VSCode 的同学安装 g++ 后，在 VSCode 集成终端中检查版本、编译并运行 Hello World，即完成本环节。使用 C++17 标准。

尚未完成 VSCode 配置的同学，使用原来的 CLion 方案：CLion 中文指南 · CLion English guide。

1. 按操作系统安装 g++

本指南面向已经安装并能正常使用 VSCode 的同学。保留现有配置，按自己的操作系统安装编译器。**安装 VSCode 或 C/C++ 扩展不等于安装了 g++**。如果已经在 VSCode 终端中运行 `g++ -v` 成功，可以直接前往第 2 节的 Hello World 验收。

选择菜单 **终端 → 新建终端（Terminal → New Terminal）**，下方打开的 Terminal 就是本文执行检查、编译和运行命令的位置。不要在“输出（Output）”或“调试控制台（Debug Console）”中输入命令。Windows 使用 PowerShell，macOS/Linux 使用 zsh 或 bash。

Windows

参考视频：Windows 下的 C++ 环境配置教程。已安装 VSCode 的同学重点完成编译器安装和 PATH 配置，再回到本文验收。

以下给出 Windows x64 使用 MSYS2 安装 g++ 的步骤：

1. 从 MSYS2 官网 下载并安装 MSYS2。以下以默认安装目录 `C:\msys64` 为例。

2. 在开始菜单打开 **MSYS2 UCRT64** 终端，执行：

```
pacman -S --needed mingw-w64-ucrt-x86_64-gcc
```

出现确认提示时按回车或输入 `Y`，等待安装完成。这条安装命令在 MSYS2 UCRT64 中执行。

3. 在 Windows 搜索中打开“编辑账户的环境变量”，编辑用户变量 `Path`，新增 `C:\msys64\ucrt64\bin`。如果修改过安装位置，请填写对应的 `ucrt64\bin` 路径。

4. 保存环境变量，完全退出所有 VSCode 窗口，再重新打开。选择菜单 **终端 → 新建终端（Terminal → New Terminal）**，使用 PowerShell，执行：

```
g++ -v
```

能看到 GCC 版本和编译器信息，就继续完成下方的 Hello World。若提示找不到 g++，检查 PATH 是否指向包含 `g++.exe` 的目录，并确认已重启 VSCode。

macOS：安装 Xcode Command Line Tools

macOS 使用 Xcode Command Line Tools 提供的编译器。在 VSCode 的集成终端中执行：

```
xcode-select --install
```

按照弹窗提示完成安装，等待安装结束。如果提示开发工具已经安装，直接继续下一步。无需安装完整的 Xcode 应用。

在 VSCode 中新建终端，执行：

```
g++ -v
```

看到 Apple Clang 版本信息即正常：macOS 的 g++ 命令实际调用 Apple Clang，可以按本指南使用 `g++ -std=c++17` 编译课程程序，无需另外安装 GNU GCC。

Linux：先检查，再按需安装

部分 Linux 环境已经安装 g++，但不能假定所有发行版都自带。在 VSCode 的集成终端中先执行：

```
g++ -v
```

如果已经显示编译器版本信息，直接进入第 2 节；如果提示找不到命令，按自己的发行版选择一组安装命令。

Ubuntu / Debian：

```
sudo apt update
sudo apt install build-essential
```

Fedora：

```
sudo dnf install gcc-c++
```

等待安装完成，在 VSCode 终端中再次执行 `g++ -v`，确认可以显示 GCC 版本信息。

2. 在 VSCode 终端编译运行 Hello World

完成编译器安装后，需要完成本节验收。以下所有检查、编译和运行命令均在 **VSCode 的集成终端** 中执行。本环节手动编译运行，无需配置 `tasks.json`、`launch.json` 或安装 Code Runner。

1. 打开练习文件夹并检查编译器

在本机新建练习文件夹，例如 `cpp-hello`，通过 VSCode 的 **文件 → 打开文件夹（File → Open Folder）** 打开它，新建终端，执行：

```
g++ -v
```

确认终端显示编译器版本信息，没有“找不到命令”之类的错误。

2. 创建并保存 hello.cpp

在当前打开的文件夹中新建 `hello.cpp`，输入下面的代码并保存，也可以直接使用附带的 hello.cpp：

```
#include <iostream>

int main() {
    std::cout << "Hello World\n";
    return 0;
}
```

确保文件名是 `hello.cpp`，而不是 `hello.cpp.txt`，并且终端当前目录就是保存该文件的文件夹。如果文件刚修改过，先保存再编译。

3. 编译并运行

Windows（PowerShell）： 分别执行下面两行。第一行成功后再运行第二行。

```
g++ -std=c++17 hello.cpp -o hello.exe
.\hello.exe
```

macOS / Linux： 分别执行下面两行。第一行成功后再运行第二行。

```
g++ -std=c++17 hello.cpp -o hello
./hello
```

编译成功时通常没有输出；运行后应在同一个终端看到：

```
Hello World
```

如果报错找不到 `hello.cpp`，用 Windows 的 `Get-Location`、`dir` 或 macOS/Linux 的 `pwd`、`ls` 检查当前目录，切换到源码所在文件夹后重新编译。

4. 完成标准

☐ 能在 VSCode 的集成终端中执行 `g++ -v` 并看到编译器版本信息。

☐ 能在同一终端中用 `g++ -std=c++17` 成功编译 `hello.cpp`。

☐ 能运行生成的程序，并在终端看到 `Hello World`。

CLion 本地 C++ 环境配置（Windows / macOS）

每位同学使用自己的学校邮箱申请 JetBrains Student Pack 教育授权，再用获得授权的 JetBrains 账号激活 CLion。准备好本地环境后，运行一个 Hello World 即完成配置。

1. 用学校邮箱申请学生授权

第一步：准备学校邮箱

确认自己能正常登录学校邮箱并收信。申请时填写完整学校邮箱地址，不是只填写学号；后续的验证和授权通知需要在邮箱中处理。

第二步：打开 Student Pack 页面并点击 Apply

打开 JetBrains Student Pack 学生申请页面，点击页面上的 Apply 或 Apply now，进入学生授权申请流程。

如果页面先要求登录 JetBrains Account：已有账号就登录；没有账号则按页面提示注册，建议使用学校邮箱。完成后继续原来的学生申请。注册账号本身还不等于已经取得学生授权。

第三步：填写学生申请

按页面提示选择学生身份，使用学校邮箱进行申请或资格验证，并如实填写要求的姓名、学校等信息。核对学校邮箱拼写后提交。如果页面要求补充在读信息，按其提示完成。

第四步：处理学校邮箱中的邮件

提交后打开学校邮箱，查看 JetBrains 发来的验证邮件并点击其中的确认链接。没有看到邮件时，检查垃圾邮件，并确认申请时填写的邮箱是否正确。

继续按照邮件和网页提示完成申请。邮箱验证后如果页面仍显示审核中，就等待审核结果；不要把“已提交申请”当成“授权已经生效”。

第五步：确认授权已关联到账号

收到通过或激活通知后，按邮件链接登录 JetBrains Account，完成授权关联。确认账号中已有可用于 CLion 的学生教育授权。

记住这个 JetBrains 账号，后面 CLion 里要登录同一个账号。若申请过程中使用了已有账号，授权可能关联在该账号下，不要激活时又换成另一账号。

2. 下载并安装 CLion

打开 CLion 官方下载页，选择电脑对应的系统和安装包。

Windows： 下载 Windows 安装程序，打开后按安装向导完成安装。安装结束后启动 CLion。

macOS： 先在苹果菜单的“关于本机”查看芯片。M 系列芯片选择 Apple Silicon，Intel 处理器选择 Intel。下载后打开安装镜像，把 CLion 拖到 Applications（应用程序）文件夹，再从该文件夹启动。

3. 在 CLion 中激活学生授权

1. 首次启动出现授权窗口时，选择登录 JetBrains Account。

2. 登录第 1 节中已经取得学生授权的那个账号，按浏览器提示完成登录并返回 CLion。

3. 在可用授权中选择学生教育授权，按界面提示激活。

4. 如果已经进入欢迎页，可从 Manage Licenses 打开授权窗口；已经打开项目时，可从 Help > Manage Subscriptions 查找。不同版本的菜单名称可能略有不同。

如果显示没有可用授权，先核对登录账号、查看申请是否获批，再尝试刷新授权列表。此时应先完成授权申请和关联，而不是反复重新安装软件。

4. 准备编译工具

Windows： 使用 CLion 内置 MinGW。若首次打开时要求选择工具链，选择 MinGW，再选择 Bundled MinGW，等待编译器自动识别。若没有自动弹出，可在设置中搜索 Toolchains 找到该选项。

macOS： 打开系统“终端”，执行下面的命令，并在弹窗中完成 Command Line Tools 安装。等待安装结束后再回到 CLion；已经安装过则跳过。

```
xcode-select --install
```

安装完成后，CLion 使用自动识别的 Apple Clang。

5. 新建程序，运行 Hello World

1. 在 CLion 欢迎页点击 New Project。

2. 选择 C++ Executable，语言标准选择 C++17，选择一个本地文件夹并创建项目。

3. 等待项目加载完成，打开自动生成的 main.cpp，把原内容替换为：

```
#include <iostream>

int main() {
    std::cout << "Hello World\n";
    return 0;
}
```

也可以复制本指南附带的 hello.cpp 中的代码。

点击 main 函数旁的绿色运行按钮，或选择当前项目后点击右上角 Run。第一次运行会先编译，等待它完成。

在下方 Run 窗口看到下面这一行，就说明本地环境配置完成：

```
Hello World
```

本程序不需要输入任何数据。

参考入口：Student Pack 申请、CLion 授权激活、Windows 编译器配置、macOS 开发工具。